obtained by iterating over expression.

2. Arguments of the form **expression are internally replaced by a sequence of keyword arguments whose names and values are obtained by iterating over expression’s items().

3. Say that the function has N positional parameters and the call has M positional arguments:

1. When M≤N, bind all the positional arguments to the first M positional parameter names; the remaining positional parameters, if any, must be matched by named arguments.

2. When M>N, bind the remaining positional arguments to named parameters in the order in
which they appear in the signature. This process terminates in one of three ways:

a. All positional arguments have been bound.

b. The next item in the signature is a * argument collector: the interpreter raises a
TypeError exception.

c. The next item in the signature is a *name argument collector: the remaining positional
arguments are collected in a tuple that is then bound to name in the function call name-
space.

4. The named arguments are then matched, in the order of the arguments’ occurrences in the call, by name with the parameters—both positional and named. Attempts to rebind an already bound pa-rameter name raise a TypeError exception.

5. If unmatched named arguments remain at this stage:

1. When the function signature includes a **name collector, the interpreter creates a dictionary
with key/value pairs (argument's_name, argument's_value), and binds it to name in the
function call namespace.

2. In the absence of such an argument collector, Python raises a TypeError exception.

6. Any remaining unmatched named parameters are bound to their default values.

At this point, the function call namespace is fully populated, and the interpreter executes the function’s body using that “call namespace” as the local namespace for the function.

The semantics of argument passing

In traditional terms, all argument passing in Python is by value (although, in modern terminology, to say that argument passing is by object reference is more precise and accurate; check out the synonym call by sharing). When you pass a variable as an argument, Python passes to the function the object (value) to which the variable currently refers (not “the variable itself”!), binding this object to the parameter name in the function call namespace. Thus, a function cannot rebind the caller’s variables. Passing a mutable object as an argument, however, allows the function to make changes to that object, because Python passes a reference to the object itself, not a copy. Rebinding a variable and mutating an object are totally disjoint concepts. For example:

print shows that a is still bound to 77. Function f’s rebinding of its parameter x to 23 has no effect on f’s caller, nor, in particular, on the binding of the caller’s variable that happened to be used to pass 77 as the parameter’s value. However, print also shows that b is now bound to [99, 42]. b is still bound to the same list object as before the call, but f has appended 42 to that list object, mutating it. In neither case has f altered the caller’s bindings, nor can f alter the number 77, since numbers are immutable. f can alter a list object, though, since list objects are mutable.

Namespaces

A function’s parameters, plus any names that are bound (by assignment or by other binding statements, such as def) in the function body, make up the function’s local namespace, also known as its local scope. Each of these variables is known as a local variable of the function.

Variables that are not local are known as global variables (in the absence of nested function definitions, which we’ll discuss shortly). Global variables are attributes of the module object, as covered in “Attributes of module objects”. Whenever a function’s local variable has the same name as a global variable, that name, within the function body, refers to the local variable, not the global one. We express this by saying that the local variable hides the global variable of the same name throughout the function body.

The global statement

By default, any variable bound in a function body is local to the function. If a function needs to bind or rebind some global variables (not best practice!), the first statement of the function’s body must be:

where identifiers is one or more identifiers separated by commas (,). The identifiers listed in a global statement refer to the global variables (i.e., attributes of the module object) that the function needs to bind or rebind. For example, the function counter that we saw in “Other attributes of function objects” could be implemented using global and a global variable, rather than an attribute of the function object:

Without the global statement, the counter function would raise an UnboundLocalError exception when called, because _count would then be an uninitialized (unbound) local variable. While the global state-ment enables this kind of programming, this style is inelegant and ill-advised. As we mentioned earlier, when you want to group together some state and some behavior, the object-oriented mechanisms cov-ered in Chapter 4 are usually best.

ESCHEW GLOBAL

Never use global if the function body just uses a global variable (including mutating the object bound to that vari-able, when the object is mutable). Use a global statement only if the function body rebinds a global variable (gener-ally by assigning to the variable’s name). As a matter of style, don’t use global unless it’s strictly necessary, as its presence causes readers of your program to assume the statement is there for some useful purpose. Never use global except as the first statement in a function body.

Nested functions and nested scopes

A def statement within a function body defines a nested function, and the function whose body includes the def is known as an outer function to the nested one. Code in a nested function’s body may access (but not rebind) local variables of an outer function, also known as free variables of the nested function.

The simplest way to let a nested function access a value is often not to rely on nested scopes, but rather to pass that value explicitly as one of the function’s arguments. If need be, you can bind the argument’s value at nested-function def time: just use the value as the default for an optional argument. For exam-ple:

Here’s the same functionality using nested scopes:

In this specific case, percent1 has one tiny advantage: the computation of a+b+c happens only once, while percent2’s inner function pc repeats the computation three times. However, when the outer func-tion rebinds local variables between calls to the nested function, repeating the computation can be neces-sary: be aware of both approaches, and choose the appropriate one on a case-by-case basis.

A nested function that accesses values from local variables of “outer” (containing) functions is also known as a closure. The following example shows how to build a closure:

Closures are sometimes an exception to the general rule that the object-oriented mechanisms covered in the next chapter are the best way to bundle together data and code. When you specifically need to con-struct callable objects, with some parameters fixed at object construction time, closures can be simpler and more direct than classes. For example, the result of make_adder(7) is a function that accepts a single argument and returns 7 plus that argument. An outer function that returns a closure is a “factory” for members of a family of functions distinguished by some parameters, such as the value of the argument augend in the previous example, which may often help you avoid code duplication.

The nonlocal keyword acts similarly to global, but it refers to a name in the namespace of some lexically surrounding function. When it occurs in a function definition nested several levels deep (a rarely needed structure!), the compiler searches the namespace of the most deeply nested containing function, then the function containing that one, and so on, until the name is found or there are no further containing func-

Các đối số có dạng * expression sẽ được thay thế bằng một chuỗi các đối số được lấy ra từ việc lặp qua các phần
tử trong expression .

Các đối số có dạng ** expression sẽ được thay thế bằng một chuỗi các đối số khóa. Tên và giá trị của các đối số
khóa này được xác định bằng cách lặp qua các phần tử trong expression ’s items() .

Giả sử hàm đó có N tham số vị trí, và lời gọi hàm đó cung cấp M đối số vị trí. Khi M≤N, hãy gán tất cả các đối số vị trí cho M tham số vị trí đầu tiên. Các đối số vị trí còn lại, nếu có, phải

được gán bằng các đối số có tên tương ứng.

Khi M > N, hãy gán các đối số còn lại vào các tham số có tên, theo thứ tự chúng xuất hiện trong định nghĩa
hàm. Quy trình này kết thúc theo một trong ba cách sau:

Tất cả các đối số vị trí đều đã được gán giá trị. Mục tiếp theo trong phần ký hiệu này là bộ thu thập các tham số dạng * . Trình thông dịch sẽ gây ra

ngoại lệ dạng TypeError .

Mục tiếp theo trong cú pháp ký hiệu này là bộ sưu tập các tham số kiểu * name . Các tham số còn lại
sẽ được thu thập vào một cấu trúc dữ liệu kiểu tuple; cấu trúc này sau đó sẽ được gán cho biến name
trong không gian tên của lời gọi hàm.

Các đối số có tên sẽ được so sánh với các tham số tương ứng, theo thứ tự mà chúng xuất hiện trong lệnh gọi hàm.
Cả các đối số được truyền theo vị trí lẫn các đối số có tên đều được xem xét. Nếu cố gắng đặt lại tên cho một tham
số đã được đặt tên trước đó, sẽ xảy ra ngoại lệ TypeError .

Nếu vẫn còn các đối số có tên chưa được khớp ở giai đoạn này: Khi thông số hàm chứa yếu tố **name collector, trình thông dịch sẽ tạo ra một từ điển gồm các cặp khóa/giá

trị dạng (argument's_name, argument's_value) . Từ điển này sau đó được gắn vào biến name trong
không gian tên của lời gọi hàm.

Trong trường hợp không có công cụ thu thập các lập luận như vậy, Python sẽ gây ra ngoại lệ TypeError . Bất kỳ tham số có tên nào vẫn chưa được gán giá trị sẽ được gán theo giá trị mặc định của chúng. Ở giai đoạn này, không gian tên dùng để gọi hàm đã được khai báo đầy đủ. Trình thông dịch sẽ thực thi nội dung của hàm,

bằng cách sử dụng “không gian tên dùng để gọi hàm” đó làm không gian tên cục bộ cho hàm đó.

Ý nghĩa ngữ nghĩa của việc truyền tham số trong các lập luận Theo quan niệm truyền thống, việc truyền các tham số trong Python đều được thực hiện dựa trên giá trị của các biến đó

(mặc dù, theo cách diễn đạt hiện đại hơn, việc truyền tham số dựa trên tham chiếu đến đối tượng cụ thể sẽ chính xác hơn; hãy xem thêm thuật ngữ “gọi hàm bằng cách chia sẻ tham chiếu”). Khi bạn truyền một biến làm tham số, Python sẽ truyền đến hàm đối tượng mà biến đó đang trỏ tới (chứ không phải chính biến đó!), và đối tượng này sẽ được gán cho tên tham số tương ứng trong hàm. Do đó, hàm không thể thay đổi giá trị của các biến được truyền vào từ bên ngoài. Tuy nhiên, nếu bạn truyền một đối tượng có thể thay đổi làm tham số, hàm có thể thay đổi giá trị của đối tượng đó, vì Python truyền tham chiếu trực tiếp đến đối tượng đó, chứ không phải bản sao của nó. Việc thay đổi giá trị của biến và việc thay đổi giá trị của đối tượng là hai khái niệm hoàn toàn khác nhau. Ví dụ:

def f(x, y):
x = 23

y.append(42)

a = 77

b = [99]
f(a, b)
print(a, b) # prints: 77 [99, 42]

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

print cho thấy rằng a vẫn còn được gắn kết với 77 . Việc f gắn lại tham số x của mình với 23 không ảnh hưởng gì đến bên gọi hàm f , cũng như không ảnh hưởng đến cách mà biến của bên gọi hàm được sử dụng để truyền giá trị 77 vào hàm. Tuy nhiên, print cho thấy rằng b hiện đang được gắn kết với [99, 42] . b vẫn còn được gắn kết với cùng một đối tượng danh sách như trước khi gọi hàm. Còn f thì đã thêm 42 vào đối tượng danh sách đó, làm thay đổi cấu trúc của đối tượng danh sách đó. Trong cả hai trường hợp, f đều không làm thay đổi các mối liên kết của bên gọi hàm. f cũng không thể thay đổi giá trị của 77 , vì các con số là những giá trị không thể thay đổi được. Tuy nhiên, f có thể thay đổi đối tượng danh sách, vì đối tượng danh sách là những đối tượng có thể thay đổi được.

Không gian tên Các tham số của một hàm, cùng với tất cả các tên biến được khai báo bên trong thân hàm (thông qua việc gán giá trị hoặc

các câu lệnh khai báo khác, chẳng hạn như def), tạo nên không gian tên cục bộ của hàm đó. Mỗi biến như vậy được gọi là biến cục bộ của hàm.

Các biến không mang tính cục bộ được gọi là biến toàn cục (trong trường hợp không có các hàm con; chúng ta sẽ thảo luận về điều này sau). Các biến toàn cục là những thuộc tính của đối tượng module, như đã được đề cập trong mục “Các thuộc tính của đối tượng module”. Mỗi khi biến cục bộ của một hàm có cùng tên với một biến toàn cục, thì trong thân hàm đó, tên đó sẽ chỉ đến biến cục bộ, chứ không phải biến toàn cục. Chúng ta có thể nói rằng biến cục bộ “che giấu” biến toàn cục có cùng tên trong suốt thân hàm đó.

Tuyên bố toàn cầu Theo mặc định, bất kỳ biến nào được khai báo bên trong thân hàm đều chỉ có hiệu lực trong phạm vi của hàm đó. Nếu hàm

cần truy cập hoặc gán lại giá trị cho một số biến toàn cục (điều này không phải là cách làm tốt nhất!), thì câu lệnh đầu tiên trong thân hàm phải là lệnh dùng để khai báo các biến toàn cục đó.

global identifiers

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Ở đây, identifiers là một hoặc nhiều định danh được phân cách bằng dấu phẩy (,). Các định danh được liệt kê trong câu lệnh global chính là các biến toàn cục (tức là các thuộc tính của đối tượng mô-đun). Hàm counter mà chúng ta đã đề cập trong phần “Các thuộc tính khác của đối tượng hàm” có thể được triển khai bằng cách sử dụng global và các biến toàn cục, thay vì sử dụng các thuộc tính của đối tượng hàm.

_count = 0
def counter():
global _count
_count += 1
return _count

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Nếu không có câu lệnh global , thì hàm counter sẽ gây ra ngoại lệ UnboundLocalError khi được gọi. Lý do là vì biến cục bộ _count lúc đó chưa được khởi tạo (chưa được gán giá trị). Mặc dù câu lệnh global cho phép việc lập trình theo cách này, nhưng cách lập trình này không hiệu quả và không nên được sử dụng. Như chúng tôi đã đề cập trước đó, khi muốn tổ chức các thông tin và hành vi liên quan với nhau, cách tốt nhất là sử dụng các cơ chế lập trình hướng đối tượng được trình bày trong Chương 4.

TRÁNH QUỐC TẾ

Đừng bao giờ sử dụng global nếu phần thân hàm chỉ sử dụng các biến toàn cục (bao gồm cả việc thay đổi giá trị của đối tượng được gắn với biến đó, trong trường hợp đối tượng đó có thểđược thay đổi). Hãy sử dụng câu lệnh global chỉ khi phần thân hàm thực hiện việc gán lại giá trị cho một biến toàn cục (thông thường là bằng cách gán giá trị vào tên của biến đó). Về mặt phong cách lập trình, đừng sử dụng global trừ khi thực sự cần thiết; sự hiện diện của câu lệnh này có thể khiến người đọc chương trình cho rằng nó có mục đích gì đó hữu ích. Đừng bao giờ sử dụng global ngoại trừ khi nó là câu lệnh đầu tiên trong phần thân hàm.

Hàm lồng nhau và phạm vi lồng nhau Một câu lệnh def bên trong thân hàm sẽ định nghĩa một hàm con. Hàm mà thân hàm của nó chứa câu lệnh def được gọi là

hàm bên ngoài so với hàm con đó. Các đoạn mã bên trong hàm con có thể truy cập vào các biến cục bộ của hàm bên ngoài (nhưng không thể thay đổi giá trị của chúng). Các biến này còn được gọi là các biến tự do của hàm con.

Cách đơn giản nhất để cho một hàm con có thể truy cập vào một giá trị nào đó là không cần dựa vào các phạm vi con, mà thay vào đó, hãy truyền giá trị đó trực tiếp như một trong các tham số của hàm con. Nếu cần thiết, bạn có thể gán giá trị của tham số đó ngay tại thời điểm hàm con được gọi: chỉ cần sử dụng giá trị đó làm giá trị mặc định cho tham số đó. Ví dụ:

def percent1(a, b, c):
def pc(x, total=a+b+c):
return (x*100.0) / total
print('Percentages are:', pc(a), pc(b), pc(c))

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Đây là cách thực hiện chức năng tương tự, nhưng sử dụng các phạm vi con:

def percent2(a, b, c):
def pc(x):
return (x*100.0) / (a+b+c)
print('Percentages are:', pc(a), pc(b), pc(c))

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Trong trường hợp cụ thể này, percent1 có một ưu điểm nhỏ: việc tính toán a+b+c chỉ diễn ra một lần duy nhất, trong khi hàm bên trong của percent2 , tức pc , lại phải thực hiện phép tính đó ba lần. Tuy nhiên, khi hàm bên ngoài gán lại các biến cục bộ giữa các lần gọi hàm bên trong, việc lặp lại phép tính là cần thiết. Hãy xem xét cả hai cách tiếp cận này, và chọn cách phù hợp tùy từng trường hợp cụ thể.

Hàm con được định nghĩa bên trong một hàm khác, và có khả năng truy cập vào các giá trị của các biến cục bộ thuộc hàm bên ngoài, còn được gọi là “closure”. Ví dụ sau minh họa cách tạo ra một closure:

def make_adder(augend):
def add(addend):
return addend+augend
return add

add5 = make_adder(5)
add9 = make_adder(9)

print(add5(100)) # prints: 105
print(add9(100)) # prints: 109

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Đôi khi, việc sử dụng các “closure” là ngoại lệ đối với quy tắc chung rằng các cơ chế lập trình hướng đối tượng được đề cập trong chương tiếp theo chính là cách tốt nhất để kết hợp dữ liệu và mã nguồn lại với nhau. Khi bạn cần tạo ra các đối tượng có thể được gọi lại, trong đó một số tham số được xác định ngay lúc tạo đối tượng, thì việc sử dụng các “closure” sẽ đơn giản và hiệu quả hơn so với việc sử dụng các lớp. Ví dụ, kết quả của make_adder(7) là một hàm nhận một tham số duy nhất và trả về giá trị của 7 cộng với tham số đó. Một hàm bên ngoài trả về một “closure” chính là “nhà máy” để tạo ra các hàm thuộc cùng một nhóm hàm, với các tham số khác nhau. Điều này thường giúp tránh việc lặp lại mã nguồn.

tions, in which case the compiler raises an error (a global name, if any, does not match).

Here’s a nested functions approach to the “counter” functionality we implemented in previous sections using a function attribute, then a global variable:

A key advantage of this approach versus the previous ones is that these two nested functions, just like an object-oriented approach would, let you make independent counters—here c1 and c2. Each closure keeps its own state and doesn’t interfere with the other one.

lambda Expressions

If a function body is a single return expression statement, you may (very optionally!) choose to replace the function with the special lambda expression form:

A lambda expression is the anonymous equivalent of a normal function whose body is a single return statement. The lambda syntax does not use the return keyword. You can use a lambda expression wher-ever you could use a reference to a function. lambda can sometimes be handy when you want to use an extremely simple function as an argument or return value.

Here’s an example that uses a lambda expression as an argument to the built-in sorted function (covered in Table 8-2):

Alternatively, you can always use a local def statement to give the function object a name, then use this name as an argument or return value. Here’s the same sorted example using a local def statement:

While lambda can at times be handy, def is usually better: it’s more general and helps you make your code more readable, since you can choose a clear name for the function.

Generators

When the body of a function contains one or more occurrences of the keyword yield, the function is known as a generator, or more precisely a generator function. When you call a generator, the function body does not execute. Instead, the generator function returns a special iterator object, known as a gener-ator object (sometimes, quite confusingly, also called just “a generator”), wrapping the function body, its local variables (including parameters), and the current point of execution (initially, the start of the func-tion).

When you (implicitly or explicitly) call next on a generator object, the function body executes from the current point up to the next yield, which takes the form:

A bare yield without the expression is also legal, and equivalent to yield None. When yield executes, the function execution is “frozen,” preserving the current point of execution and local variables, and the ex-pression following yield becomes the result of next. When you call next again, execution of the function body resumes where it left off, again up to the next yield. When the function body ends, or executes a return statement, the iterator raises a StopIteration exception to indicate that the iteration is finished. The expression after return, if any, is the argument to the StopIteration exception.

yield is an expression, not a statement. When you call g.send(value) on a generator object g, the value of yield is value; when you call next(g), the value of yield is None. We’ll talk more about this shortly: it’s an elementary building block for implementing coroutines in Python.

A generator function is often a handy way to build an iterator. Since the most common way to use an iter-ator is to loop on it with a for statement, you typically call a generator like this (with the call to next be-ing implicit in the for statement):

For example, say that you want a sequence of numbers counting up from 1 to N and then down to 1 again. A generator can help:

Here is a generator that works somewhat like built-in range, but returns an iterator on floating-point val-ues rather than on integers:

This example is only somewhat like range because, for simplicity, it makes the arguments start and stop mandatory, and assumes that stride is positive.

Generator functions are more flexible than functions that return lists. A generator function may return an unbounded iterator, meaning one that yields an infinite stream of results (to use only in loops that ter-minate by other means, e.g., via a conditionally executed break statement). Further, a generator object it-erator performs lazy evaluation: the iterator can compute each successive item only when and if needed, “just in time,” while the equivalent function does all computations in advance and may require large amounts of memory to hold the results list. Therefore, if all you need is the ability to iterate on a com-puted sequence, it is usually best to compute the sequence in a generator object, rather than in a function returning a list. If the caller needs a list of all the items produced by some bounded generator object built by g(arguments), the caller can simply use the following code to explicitly request that Python build a list:

yield from

To improve execution efficiency and clarity when multiple levels of iteration are yielding values, you can use the form yield from expression, where expression is iterable. This yields the values from expression one at a time into the calling environment, avoiding the need to yield repeatedly. We can thus simplify the updown generator we defined earlier:

Moreover, using yield from lets you use generators as coroutines, discussed next.

Generators as near-coroutines

Generators are further enhanced with the possibility of receiving a value (or an exception) back from the caller as each yield executes. This lets generators implement coroutines, as explained in PEP 342. When a generator object resumes (i.e., you call next on it), the corresponding yield’s value is None. To pass a value x into some generator object g (so that g receives x as the value of the yield on which it’s sus-pended), instead of calling next(g), call g.send(x) (g.send(None) is just like next(g)).

Other enhancements to generators regard exceptions: we cover them in “Generators and Exceptions”.

Generator expressions

Python offers an even simpler way to code particularly simple generators: generator expressions, com-monly known as genexps. The syntax of a genexp is just like that of a list comprehension (as covered in “List comprehensions”), except that a genexp is within parentheses (()) instead of brackets ([]). The se-mantics of a genexp are the same as those of the corresponding list comprehension, except that a genexp produces an iterator yielding one item at a time, while a list comprehension produces a list of all results in memory (therefore, using a genexp, when appropriate, saves memory). For example, to sum the squares of all single-digit integers, you could code sum([x*x for x in range(10)]), but you can express this better as sum(x*x for x in range(10)) (just the same, but omitting the brackets): you get the same re-sult but consume less memory. The parentheses that indicate the function call also do “double duty” and enclose the genexp. Parentheses are, however, required when the genexp is not the sole argument. Additional parentheses don’t hurt, but are usually best omitted, for clarity.

WARNING: DON’T ITERATE OVER A GENERATOR MULTIPLE TIMES

A limitation of generators and generator expressions is that you can iterate over them only once. Calling next on a generator that has been consumed will just raise StopIteration again, which most functions will take as an indica-tion that the generator returns no values. If your code is not careful about reusing a consumed generator, this can in-troduce bugs:

You can write code to guard against accidentally iterating over a consumed generator by using a class to wrap the generator, like the following:

Now an erroneous reuse of a generator will raise an exception:

Recursion

Python supports recursion (i.e., a Python function can call itself, directly or indirectly), but there is a limit to how deep the recursion can go. By default, Python interrupts recursion and raises a RecursionLimitExceeded exception (covered in “Standard Exception Classes”) when it detects that re-cursion has exceeded a depth of 1,000. You can change this default recursion limit by calling the setrecursionlimit function in the module sys, covered in Table 8-3.

Note that changing the recursion limit does not give you unlimited recursion. The absolute maximum limit depends on the platform on which your program is running, and particularly on the underlying op-erating system and C runtime library, but it’s typically a few thousand levels. If recursive calls get too deep, your program crashes. Such runaway recursion, after a call to setrecursionlimit that exceeds the platform’s capabilities, is one of the few things that can cause a Python program to crash—really crash, hard, without the usual safety net of Python’s exception mechanism. Therefore, beware of “fixing” a pro-gram that is getting RecursionLimitExceeded exceptions by raising the recursion limit with setrecursionlimit. While this is a valid technique, most often you’re better advised to look for ways to remove the recursion unless you are confident you’ve been able to limit the depth of recursion that your program needs.

Readers who are familiar with Lisp, Scheme, or functional programming languages must in particular be aware that Python does not implement the optimization of tail-call elimination, which is so crucial in those languages. In Python, any call, recursive or not, has the same “cost” in terms of both time and mem-ory space, dependent only on the number of arguments: the cost does not change, whether the call is a “tail call” (meaning that it’s the last operation that the caller executes) or not. This makes recursion re-moval even more important.

For example, consider a classic use for recursion: walking a binary tree. Suppose you represent a binary tree structure as nodes, where each node is a three-item tuple (payload, left, right), and left and right are either similar tuples or None, representing the left-side and right-side descendants. A simple ex-ample might be (23, (42, (5, None, None), (55, None, None)), (94, None, None)) to represent the tree shown in Figure 3-1.

Figure 3-1. An example of a binary tree

To write a generator function that, given the root of such a tree, “walks” the tree, yielding each payload in top-down order, the simplest approach is recursion:

But if a tree is very deep, recursion can become a problem. To remove recursion, we can handle our own stack—a list used in last-in, first-out (LIFO) fashion, thanks to its append and pop methods. To wit:

The only small issue to be careful about, to keep exactly the same order of yields as rec, is switching the (1, 2) index order in which to examine descendants to (2, 1), adjusting to the “reversed” (last-in, first-out) behavior of stack.

Từ khóa nonlocal hoạt động tương tự như global . Tuy nhiên, từ khóa này tham chiếu đến một tên nằm trong không gian tên của một hàm nào đó. Khi từ khóa này xuất hiện trong định nghĩa của một hàm nằm sâu trong nhiều cấp độ khác nhau (một cấu trúc hiếm khi được sử dụng!), trình biên dịch sẽ tìm kiếm trong không gian tên của hàm nằm sâu nhất; sau đó là hàm chứa hàm đó, và cứ thế tiếp tục cho đến khi tìm thấy tên cần tìm, hoặc khi không còn hàm nào chứa tên đó nữa. Trong trường hợp này, trình biên dịch sẽ báo lỗi (nếu có tên toàn cục nào trùng với tên đó, thì cũng sẽ xảy ra lỗi).

Đây là cách sử dụng các hàm con để thực hiện chức năng “đếm số lần thực hiện”. Trong các phần trước, chúng ta đã sử dụng thuộc tính của hàm, cùng với biến toàn cục để thực hiện chức năng này.

def make_counter():
count = 0

def counter():
nonlocal count
count += 1
return count
return counter

c1 = make_counter()
c2 = make_counter()
print(c1(), c1(), c1()) # prints: 1 2 3
print(c2(), c2()) # prints: 1 2
print(c1(), c2(), c1()) # prints: 4 3 5

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Một ưu điểm lớn của phương pháp này so với các phương pháp trước đây là việc sử dụng hai hàm con này giúp chúng ta có thể tạo ra các bộ đếm độc lập với nhau – cụ thể là c1 và c2 . Mỗi hàm con đều giữ nguyên trạng thái của mình và không ảnh hưởng gì đến hàm con kia.

Biểu thức lambda Nếu thân hàm chỉ gồm một câu lệnh return expression , bạn có thể (tùy chọn thôi!) thay thế hàm đó bằng dạng biểu

thức đặc biệt lambda .

lambda parameters: expression

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Biểu thức lambda tương đương với một hàm ẩn danh; phần thân của biểu thức này chính là một câu lệnh return . Cú pháp lambda không sử dụng từ khóa return . Bạn có thể sử dụng biểu thức lambda ở bất cứ nơi nào mà bạn có thể sử dụng tham chiếu đến một hàm. Đôi khi, lambda rất hữu ích khi bạn muốn sử dụng một hàm cực kỳ đơn giản làm đối số hoặc giá trị trả về.

Đây là một ví dụ về việc sử dụng biểu thức lambda làm đối số cho hàm tích hợp sẵn sorted (được trình bày trong Bảng 8-2):

a_list = [-2, -1, 0, 1, 2]
sorted(a_list, key=lambda x: x * x) # returns: [0, -1, 1, -2, 2]

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Hoặc bạn cũng có thể sử dụng câu lệnh def để đặt tên cho đối tượng hàm đó. Sau đó, bạn có thể sử dụng tên này làm đối số hoặc giá trị trả về của hàm. Dưới đây là ví dụ tương tự, nhưng sử dụng câu lệnh def :

a_list = [-2, -1, 0, 1, 2]
def square(value):
return value * value

sorted(a_list, key=square) # returns: [0, -1, 1, -2, 2]

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Mặc dù lambda đôi khi cũng hữu ích, nhưng def thường tốt hơn: nó có tính ứng dụng cao hơn và giúp mã nguồn trở nên dễ đọc hơn. Bạn có thể chọn một tên rõ ràng cho hàm đó.

Máy phát điện Khi thân hàm chứa một hoặc nhiều lần xuất hiện của từ khóa yield , thì hàm đó được gọi là hàm sinh viên (generator

function). Khi gọi một hàm sinh viên, thân hàm sẽ không được thực thi. Thay vào đó, hàm sinh viên sẽ trả về một đối tượng đặc biệt, được gọi là đối tượng sinh viên (generator object). Đối tượng này bao gồm thân hàm, các biến cục bộ của hàm (bao gồm cả các tham số), và điểm thực thi hiện tại của hàm (ban đầu là điểm bắt đầu của hàm).

Khi bạn gọi hàm next trên một đối tượng generator, thì nội dung của hàm sẽ được thực thi, bắt đầu từ điểm hiện tại cho đến dấu yield tiếp theo. Cú pháp gọi hàm như sau:

yield expression

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Cú pháp yield khi không kèm theo biểu thức nào cũng được chấp nhận, và tương đương với cú pháp yield None . Khi yield được thực thi, quá trình thực thi hàm sẽ bị “đóng băng”: điểm thực thi hiện tại và các biến cục bộ sẽ được giữ nguyên. Biểu thức nằm sau yield sẽ trở thành kết quả của next . Khi gọi next một lần nữa, quá trình thực thi hàm sẽ tiếp tục từ điểm dừng trước đó, cho đến khi gặp lệnh yield tiếp theo. Khi quá trình thực thi hàm kết thúc, hoặc khi lệnh return được thực thi, bộ lặp sẽ gây ra ngoại lệ StopIteration để báo hiệu rằng vòng lặp đã kết thúc. Biểu thức nằm sau return (nếu có) sẽ trở thành tham số của ngoại lệ StopIteration .

yield là một biểu thức, chứ không phải là mệnh lệnh. Khi bạn gọi g.send(value) trên đối tượng tạo ra dữ liệu thời gian thực g , giá trị của yield sẽ bằng value . Khi bạn gọi next(g) , giá trị của yield sẽ bằng None . Chúng ta sẽ thảo luận kỹ hơn về điều này sau: đây là yếu tố cơ bản để triển khai các chương trình kiểu coroutine trong Python.

Hàm tạo ra các phần tử của một vòng lặp thường là cách hiệu quả để xây dựng các bộ lặp. Vì cách sử dụng phổ biến nhất của các bộ lặp là thực hiện các lệnh lặp bằng cú pháp for , nên người ta thường gọi hàm tạo ra các phần tử của vòng lặp theo cách này (việc gọi hàm next được thực hiện một cách ngầm định trong cú pháp for).

for avariable in somegenerator(arguments):

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Ví dụ, giả sử bạn muốn tạo một dãy số tăng dần từ 1 lên N , sau đó giảm dần xuống 1 . Bạn có thể sử dụng công cụ tạo dãy số để thực hiện việc này.

def updown(N):
for x in range(1, N):
yield x
for x in range(N, 0, -1):
yield x
for i in updown(3):
print(i) # prints: 1 2 3 2 1

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Đây là một công cụ tạo ra các giá trị số dạng số thực, hoạt động tương tự như chức năng range có sẵn trong hệ thống. Tuy nhiên, công cụ này trả về một đối tượng dùng để duyệt qua các giá trị số thực, thay vì các giá trị số nguyên.

def frange(start, stop, stride=1.0):
start = float(start) # force all yielded values to be floats
while start < stop:
yield start
start += stride

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Ví dụ này chỉ có chút tương đồng với range . Để đơn giản hóa, ví dụ này yêu cầu rằng các tham số start và stop phải được nhập vào; đồng thời giả định rằng giá trị của stride luôn dương.

Các hàm tạo ra dữ liệu theo kiểu “generator” có tính linh hoạt cao hơn so với các hàm trả về danh sách. Một hàm kiểu generator có thể trả về một bộ lặp vô hạn, tức là bộ lặp này sẽ liên tục tạo ra các giá trị một cách vô hạn (chỉ nên được sử dụng trong các vòng lặp kết thúc bằng các điều kiện nhất định, chẳng hạn như thông qua câu lệnh break được thực thi có điều kiện). Ngoài ra, bộ lặp do hàm generator tạo ra thực hiện việc tính toán một cách “từ từ”: mỗi giá trị chỉ được tính toán khi cần thiết, tức là “đúng lúc cần thiết”. Trong khi đó, hàm thông thường sẽ tính toán tất cả các giá trị trước, và điều này có thể khiến việc sử dụng bộ nhớ trở nên phức tạp. Do đó, nếu bạn chỉ cần khả năng lặp qua các giá trị được tạo ra bởi hàm generator, thì tốt nhất nên sử dụng hàm generator thay vì hàm trả về danh sách. Nếu người gọi hàm cần danh sách chứa tất cả các giá trị được tạo ra bởi hàm generator, họ có thể sử dụng đoạn mã sau để yêu cầu Python tạo ra danh sách đó:

resulting_list = list(g(arguments))

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Nhượng bộ/Chịu thua Để nâng cao hiệu quả và tính rõ ràng trong quá trình xử lý dữ liệu khi có nhiều lần lặp lại, bạn có thể sử dụng cú pháp

yield from expression . Trong đó, expression là đối tượng có thể được lặp qua. Cách này giúp truyền các giá trị từ expression vào môi trường gọi một cách từng cái một, tránh việc phải thực hiện thao tác yield nhiều lần. Nhờ đó, chúng ta có thể đơn giản hóa cấu trúc của bộ tạo giá trị updown mà chúng ta đã định nghĩa trước đó.

def updown(N):
yield from range(1, N)
yield from range(N, 0, -1)
for i in updown(3):
print(i) # prints: 1 2 3 2 1

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Hơn nữa, việc sử dụng yield from giúp chúng ta có thể sử dụng các hàm tạo giá trị một cách tương tự như các chương trình đa luồng, điều này sẽ được trình bày chi tiết ở phần sau.

Các bộ tạo xung như những “coroutine” gần như hoàn hảo. Các bộ tạo giá trị được cải tiến thêm, cho phép nhận lại một giá trị (hoặc một ngoại lệ) từ phía người gọi, mỗi khi lệnh

yield được thực thi. Điều này giúp các bộ tạo giá trị có thể thực hiện chức năng của các hàm đồng thời, như đã được giải thích trong PEP 342. Khi một đối tượng bộ tạo giá trị được tiếp tục thực thi (tức là khi lệnh next được gọi trên đối tượng đó), giá trị tương ứng của lệnh yield sẽ được lưu lại. Để truyền một giá trị x vào đối tượng bộ tạo giá trị g (để g nhận được giá trị x), thay vì gọi lệnh next(g) , hãy gọi các lệnh g và .send(x) . Lệnh (g.send(None) thì tương tự như lệnh next(g) .

Những cải tiến khác liên quan đến các hàm tạo đối tượng bao gồm cách xử lý các ngoại lệ: Chúng tôi sẽ trình bày chi tiết về điều này trong mục “Các hàm tạo đối tượng và các ngoại lệ”.

Các biểu thức tạo ra giá trị (Generator expressions) Python cung cấp cách viết mã đơn giản hơn nữa để tạo ra các “generator” đơn giản: đó chính là các biểu thức generator,

thường được gọi là genexp. Cú pháp của genexp tương tự như cú pháp của các biểu thức lập danh sách (như đã đề cập trong mục “Biểu thức lập danh sách”), chỉ khác ở chỗ genexp được đặt trong dấu ngoặc đơn () thay vì dấu ngoặc vuông []. Ý nghĩa của genexp cũng tương tự như biểu thức lập danh sách tương ứng; tuy nhiên, genexp tạo ra một iterator, tức là trả về từng phần tử một, trong khi biểu thức lập danh sách tạo ra toàn bộ danh sách các kết quả trong bộ nhớ. Do đó, việc sử dụng genexp sẽ giúp tiết kiệm bộ nhớ khi cần thiết. Ví dụ, để tính tổng bình phương của tất cả các số nguyên một chữ số, bạn có thể viết sum([x*x for x in range(10)] , nhưng cách viết sum(x*x for x in range(10)) sẽ hiệu quả hơn (vẫn cho kết quả tương tự nhưng tiết kiệm bộ nhớ hơn). Dấu ngoặc đơn dùng để chỉ việc gọi hàm cũng đồng thời dùng để bao quanh genexp.Tuy nhiên, dấu ngoặc đơn là cần thiết khi biểu thức genexp không phải là đối số duy nhất được truyền vào. Việc sử dụng thêm dấu ngoặc đơn cũng không gây ảnh hưởng gì, nhưng thường thì tốt nhất là nên bỏ qua chúng để giúp câu lệnh trở nên rõ ràng hơn.

CẢNH BÁO: KHÔNG ĐƯỢC LẶP LẠI VIỆC SỬ DỤNG BỘ TẠO SỐ NHIỀU LẦN.

Một hạn chế của các bộ tạo giá trị và các biểu thức tạo giá trị là chúng chỉ có thểđược xử lý một lần duy nhất. Việc gọi hàm next đối với một bộ tạo giá trị đã được sử dụng hết các giá trị của nó sẽ chỉ khiến hàm StopIteration được gọi lại. Hầu hết các hàm sẽ coi điều này là dấu hiệu cho thấy bộ tạo giá trị đó không còn giá trị nào để trả về nữa. Nếu mã nguồn của bạn không cẩn trọng trong việc tái sử dụng các bộ tạo giá trị đã được sử dụng hết, điều này có thể dẫn đến các lỗi trong chương trình.

# create a generator and list its items and their sum
squares = (x*x for x in range(5))
print(list(squares)) # prints [0, 1, 4, 9, 16]
print(sum(squares)) # Bug! Prints 0

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Bạn có thể viết mã để tránh tình trạng lặp lại quá trình xử lý dữ liệu từ bộ tạo dữ liệu một cách vô tình. Cách làm này là sử dụng một lớp để bao bọc bộ tạo dữ liệu đó, như sau:

class ConsumedGeneratorError(Exception):
"""Raised if a generator is accessed after
already consumed.
"""

class StrictGenerator:
"""Wrapper for generator that will only permit it
to be consumed once. Additional accesses will raise ConsumedGeneratorError.

"""

def __init__(self, gen):
self._gen = gen
self._gen_consumed = False
def __iter__(self):
return self

def __next__(self):
try:
return next(self._gen)
except StopIteration:
if self._gen_consumed:
raise ConsumedGeneratorError() from None
self._gen_consumed = True
raise

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Bây giờ, việc sử dụng lại một đối tượng generator một cách sai lầm sẽ gây ra lỗi.

squares = StrictGenerator(x*x for x in range(5))
print(list(squares)) # prints: [0, 1, 4, 9, 16]
print(sum(squares)) # raises ConsumedGeneratorError

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Lặp lại/Đệ quy Python hỗ trợ việc gọi lại chính mình (tức là một hàm trong Python có thể gọi chính nó, trực tiếp hoặc gián tiếp). Tuy nhiên,

có một giới hạn về độ sâu của việc gọi lại này. Theo mặc định, khi mức độ gọi lại vượt quá 1.000 lần, Python sẽ ngắt quá trình gọi lại đó và gây ra ngoại lệ RecursionLimitExceeded . Chi tiết về điều này có thể được tìm thấy trong mục “Các lớp ngoại lệ tiêu chuẩn”. Bạn có thể thay đổi giới hạn này bằng cách gọi hàm setrecursionlimit trong mô-đun sys . Chi tiết về điều này có thể được tìm thấy trong Bảng 8-3.

Lưu ý rằng việc thay đổi giới hạn mức độ lặp lại không có nghĩa là bạn có thể thực hiện các lệnh lặp lại vô hạn. Giới hạn tối đa phụ thuộc vào nền tảng mà chương trình của bạn đang chạy trên đó, cụ thể là hệ điều hành và thư viện thời gian chạy của ngôn ngữ C. Thông thường, giới hạn này vào khoảng vài nghìn lần lặp lại. Nếu các lệnh lặp lại diễn ra quá nhiều lần, chương trình của bạn sẽ bị treo. Hiện tượng lặp lại vô hạn như vậy, đặc biệt khi số lần lặp vượt quá khả năng xử lý của nền tảng, là một trong những nguyên nhân khiến chương trình Python bị treo hoàn toàn, mà không có cơ chế bảo vệ nào của Python có thể ngăn chặn được điều này. Do đó, hãy cẩn trọng khi cố gắng “sửa chữa” chương trình gặp lỗi do lặp lại quá nhiều lần bằng cách tăng giới hạn mức độ lặp lại. Mặc dù đây là một phương pháp hợp lý, nhưng thường thì tốt hơn hết là nên tìm cách loại bỏ việc lặp lại, trừ khi bạn chắc chắn rằng mình đã kiểm soát được mức độ lặp lại mà chương trình cần.

Những độc giả quen thuộc với các ngôn ngữ lập trình như Lisp, Scheme hoặc các ngôn ngữ lập trình hàm chắc hẳn cần lưu ý rằng Python không áp dụng kỹ thuật tối ưu hóa liên quan đến việc loại bỏ các lời gọi hàm kiểu “tail call”. Kỹ thuật này rất quan trọng trong các ngôn ngữ nói trên. Trong Python, mọi lời gọi hàm, dù có phải là lời gọi tuần hoàn hay không, đều có cùng “chi phí” về mặt thời gian và bộ nhớ. Chi phí này chỉ phụ thuộc vào số lượng đối số được truyền vào hàm. Do đó, việc loại bỏ các lời gọi hàm kiểu “tail call” càng trở nên quan trọng hơn.

Ví dụ, hãy xem xét một ứng dụng điển hình của phương pháp lặp lại trong lập trình: việc duyệt qua một cây nhị phân. Giả sử chúng ta biểu diễn cấu trúc cây nhị phân dưới dạng các nút; mỗi nút gồm ba phần tử: (payload, left, right) , left và right . Các phần tử left và right có thể là các cặp phần tử tương tự nhau, hoặc chúng có thể là None , biểu thị các nút con ở phía bên trái và bên phải của nút đó. Một ví dụ đơn giản có thể là: (23, (42, (5, None, None), (55, None, None)), (94, None, None)) , để biểu diễn cây như được minh họa trong Hình 3-1.

Hình 3-1. Ví dụ về một cây nhị phân Để viết một hàm tạo ra dãy các giá trị trong cây, bắt đầu từ gốc cây, cách đơn giản nhất là sử dụng phương pháp lặp lại

(recursion): hàm sẽ “duyệt” từng nút trong cây, và trả về giá trị của mỗi nút theo thứ tự từ trên xuống dưới.

def rec(t):
yield t[0]
for i in (1, 2):
if t[i] is not None:
yield from rec(t[i])

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Nhưng nếu một cây có cấu trúc sâu, việc sử dụng phương pháp lặp lại có thể gây ra các vấn đề. Để khắc phục tình trạng này, chúng ta có thể tự quản lý “stack” của mình – một danh sách được sử dụng theo nguyên tắc “cuối vào, đầu ra” (LIFO). Điều này có thể được thực hiện nhờ vào các phương thức append và pop của danh sách đó.

def norec(t):
stack = [t]
while stack:
t = stack.pop()
yield t[0]
for i in (2, 1):
if t[i] is not None:
stack.append(t[i])

Explain this code

Hãy giải thích đoạn mã này.

Translate into

Dịch sang tiếng Việt:

Điểm cần lưu ý duy nhất là phải đảm bảo rằng thứ tự của các yield vẫn giống hệt thứ tự của các rec . Điều này đòi hỏi phải thay đổi thứ tự xử lý các (1, 2) , sao cho chúng được xử lý theo nguyên tắc “từ sau ra trước” của stack .

.